

TASK - 8

(laptop as server and 2 esp32 is generating 5 random telemetry and the data is sent to laptop using mqtt protocol)

TASK :

Two Esp32 were used and each esp32 is coded such that both of them randomly generate 5 given telemetries like temperature, moisture, lighting etc.. and these data is sent to laptop which act as server using mqtt protocol ie, we can send the data of multiple devices connected to multiple sensors to the server using mqtt server

Instead of sending 5 separate messages for each sensor, all 5 telemetry readings into a single **JSON package**

The MQTT Topic Architecture

To keep the data from the two ESP32s separate, we will use two distinct MQTT channels (topics):

- **ESP32 #1** will publish to: group_project/esp1
- **ESP32 #2** will publish to: group_project/esp2

laptop will subscribe to both at the same time.

Step 1: Program ESP32 Node 1 (Thonny)

Connect **first ESP32** to Thonny. This script uses the urandom library to generate random numbers to simulate 5 telemetry parameters (Light, Battery, Temperature, Distance, and Moisture).

Create a new file in Thonny and save it as main.py on **first ESP32**:

```
import machine
import time
import network
import urandom    # For generating random telemetry data
import json
from umqtt.simple import MQTTClient

# Wi-Fi & MQTT Configuration
WIFI_SSID = "YOUR_WIFI_NAME"
WIFI_PASSWORD = "YOUR_WIFI_PASSWORD"
MQTT_BROKER = "broker.hivemq.com"
MQTT_PORT = 1883
MQTT_TOPIC = b"group_project/esp1" # Topic for ESP 1
CLIENT_ID = "ESP32_Node_1"

# Connect to Wi-Fi
```

```

wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect(WIFI_SSID, WIFI_PASSWORD)

print("ESP1: Connecting to Wi-Fi...")
while not wlan.isconnected():
    time.sleep(1)
print("ESP1: Connected!")

# Connect to MQTT Broker
print("ESP1: Connecting to MQTT...")
try:
    client = MQTTClient(CLIENT_ID, MQTT_BROKER, port=MQTT_PORT)
    client.connect()
    print("ESP1: Connected to Broker successfully!")
except Exception as e:
    print("ESP1: Broker connection failed:", e)

# Main Loop: Generate and send telemetry every 3 seconds
while True:
    try:
        # Simulate 5 telemetries with realistic random ranges
        temp = round(urandom.uniform(20.0, 40.0), 2)      # 20°C to
40°C
        dist = round(urandom.uniform(10.0, 200.0), 2)     # 10cm to
200cm
        moist = round(urandom.uniform(30.0, 90.0), 2)    # 30% to 90%
        light = round(urandom.uniform(0.0, 100.0), 2)    # 0% to 100%
        batt = round(urandom.uniform(0.0, 100.0), 2)     # 0% to 100%

        # Package everything beautifully into a single JSON
dictionary
        telemetry_data = {
            "temp": temp,
            "dist": dist,
            "moist": moist,
            "light": light,
            "batt": batt
        }

        # Convert dictionary to JSON string
        payload = json.dumps(telemetry_data)
        print("ESP1 Sending:", payload)

        # Publish payload
        client.publish(MQTT_TOPIC, payload)

    except Exception as e:

```

```
print("ESP1 Error:", e)

time.sleep(3)
```

Step 2: Program ESP32 Node 2 (Thonny)

plug in **second ESP32** to Thonny, and flash this code.

Instead of removing the first esp32 we can create an another file and select the other port in thonny (bottom right) where second esp32 is connected and in the new file we can paste this code

Save – ctrl+shift+s

Two options will be shown

a)Micropython device

b) This computer

select micropython device and save it in .py format (eg: [boot.py](#), [main.py](#))

Note: The only differences here are the CLIENT_ID and the MQTT_TOPIC, which is now esp2.

```
import machine
import time
import network
import urandom
import json
from umqtt.simple import MQTTClient

# Wi-Fi & MQTT Configuration
WIFI_SSID = "YOUR_WIFI_NAME"
WIFI_PASSWORD = "YOUR_WIFI_PASSWORD"
MQTT_BROKER = "broker.hivemq.com"
MQTT_PORT = 1883
MQTT_TOPIC = b"group_project/esp2" # Topic for ESP 2
CLIENT_ID = "ESP32_Node_2"

# Connect to Wi-Fi
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect(WIFI_SSID, WIFI_PASSWORD)

print("ESP2: Connecting to Wi-Fi...")
while not wlan.isconnected():
    time.sleep(1)
print("ESP2: Connected!")

# Connect to MQTT Broker
print("ESP2: Connecting to MQTT...")
try:
```

```

    client = MQTTClient(CLIENT_ID, MQTT_BROKER, port=MQTT_PORT)
    client.connect()
    print("ESP2: Connected to Broker successfully!")
except Exception as e:
    print("ESP2: Broker connection failed:", e)

# Main Loop
while True:
    try:
        # Simulate 5 telemetries
        temp = round(urandom.uniform(20.0, 40.0), 2)
        dist = round(urandom.uniform(10.0, 200.0), 2)
        moist = round(urandom.uniform(30.0, 90.0), 2)
        light = round(urandom.uniform(0.0, 100.0), 2)
        batt = round(urandom.uniform(0.0, 100.0), 2)

        telemetry_data = {
            "temp": temp,
            "dist": dist,
            "moist": moist,
            "light": light,
            "batt": batt
        }

        payload = json.dumps(telemetry_data)
        print("ESP2 Sending:", payload)
        client.publish(MQTT_TOPIC, payload)

    except Exception as e:
        print("ESP2 Error:", e)

    time.sleep(3)

```

Step 3: Write the Central Receiver Code in VS Code

Now open **VS Code** on the laptop. This script will subscribe to both topics simultaneously. When a message arrives, it will look at which channel it came from and display the parameters with either the index 1 or 2.

Create a new file named `multi_node_receiver.py` and paste this code:

```

import paho.mqtt.client as mqtt
import json

MQTT_BROKER = "broker.hivemq.com"
MQTT_PORT = 1883

```

```

# Subscribing to both specific topics
TOPIC_ESP1 = "group_project/esp1"
TOPIC_ESP2 = "group_project/esp2"

def on_connect(client, userdata, flags, rc):
    if rc == 0:
        print("[*] Connected to HiveMQ Broker!")
        # Subscribe to both channels at once
        client.subscribe(TOPIC_ESP1)
        client.subscribe(TOPIC_ESP2)
        print("[*] Subscribed to ESP1 and ESP2 topics. Waiting for
telemetries...\n")
    else:
        print(f"[-] Connection failed with code {rc}")

def on_message(client, userdata, msg):
    try:
        # Decode the byte payload into text, then parse the JSON
string back into a Python dict
        payload_text = msg.payload.decode("utf-8")
        data = json.loads(payload_text)

        # Check which ESP device sent the message based on the topic
channel
        if msg.topic == TOPIC_ESP1:
            print("===== DATA FROM ESP32 NODE 1
=====")
            print(f"Temperature (t1): {data['temp']} °C | Distance
(d1): {data['dist']} cm")
            print(f"Moisture (m1) : {data['moist']} % | Light
(l1) : {data['light']} %")
            print(f"Battery (b1) : {data['batt']} %")

print("=====")

            elif msg.topic == TOPIC_ESP2:
                print("===== DATA FROM ESP32 NODE 2
=====")
                print(f"Temperature (t2): {data['temp']} °C | Distance
(d2): {data['dist']} cm")
                print(f"Moisture (m2) : {data['moist']} % | Light
(l2) : {data['light']} %")
                print(f"Battery (b2) : {data['batt']} %")

print("=====")

        print("\n") # Adds a clean spacing between incoming reads

```

```

except Exception as e:
    print(f"[-] Error parsing incoming data: {e}")

# Setup MQTT Client
client = mqtt.Client()
client.on_connect = on_connect
client.on_message = on_message

print("[*] Connecting to broker...")
client.connect(MQTT_BROKER, MQTT_PORT, 60)

try:
    client.loop_forever()
except KeyboardInterrupt:
    print("\n[*] Disconnecting. Project stopped.")
    client.disconnect()

```

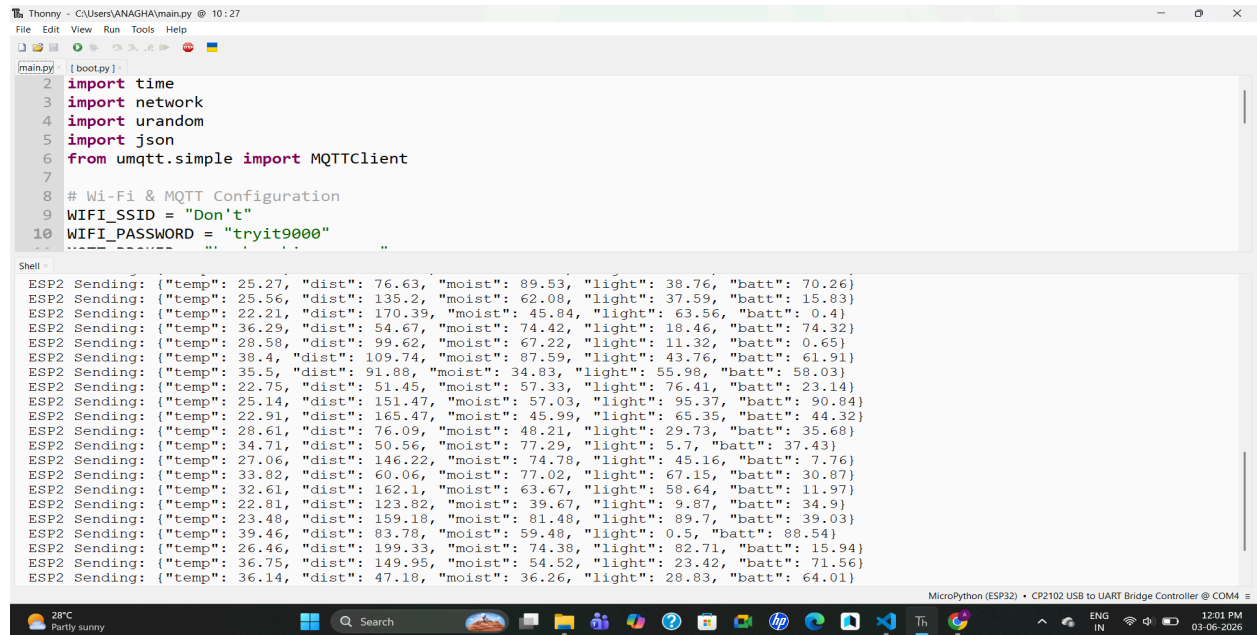
Step 4: Run the Test Experiment

1. **Power up both ESP32s:** You can plug one into your computer and plug the second one into a regular USB mobile charger adapter or another teammate's laptop so they are both powered on and running their scripts.
Here we can connect both the esp32 to each com port select one port in thonny and save the code for one esp32 and run it without stopping select the other file and also change the com port where the other esp is connected and run the code for this esp
Now these two esp's are generating random telemetries we can only see the values of any one of the esp in the shell of thonny while in the vs code both the data will appear
2. **Run Python script in VS Code.** Watch VS Code terminal. we will see well-organized telemetry boxes flashing up. Node 1 readings will display cleanly under t1, d1, m1... tags, and Node 2 data will print perfectly under t2, d2, m2... tags right next to it

OUTPUT

THONNY

ESP32 NODE 2 ([main.py](#)) { ESP32 NODE 1 code is also running in background ie, in [boot.py](#) }



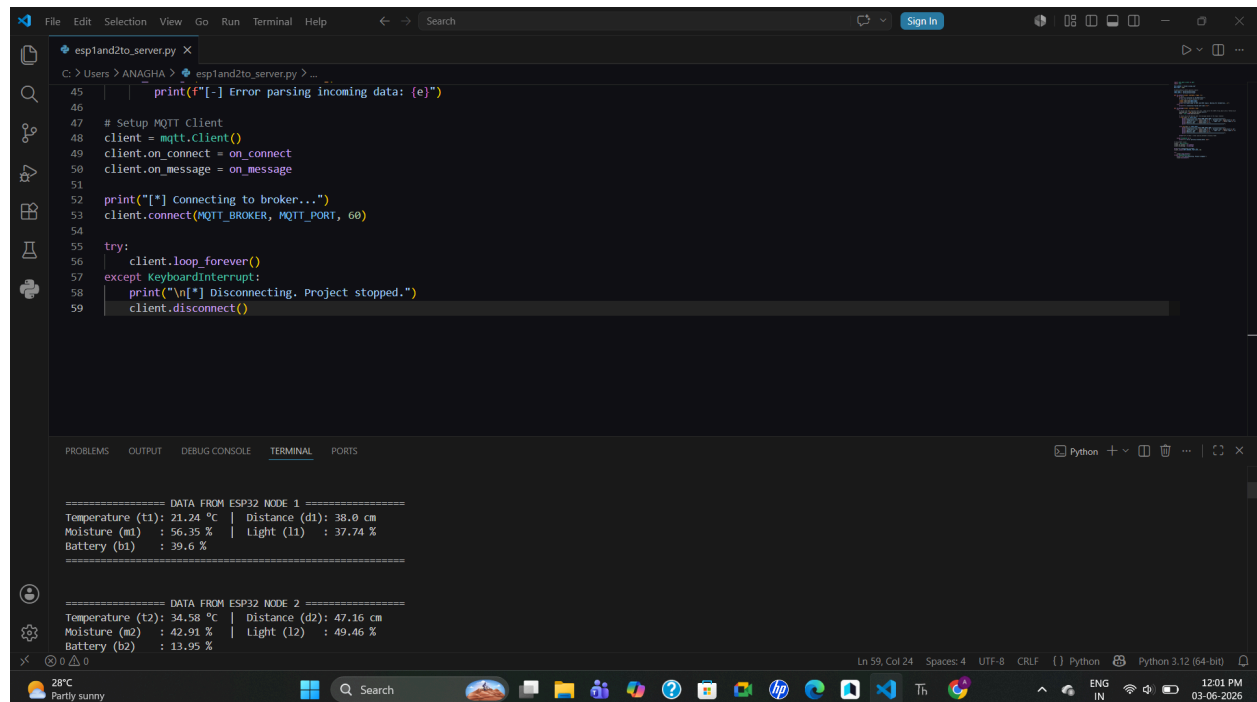
The screenshot shows the Thonny IDE interface. The main editor window displays a Python script named `main.py` with the following code:

```
1 import time
2 import network
3 import urandom
4 import json
5 from umqtt.simple import MQTTClient
6
7
8 # Wi-Fi & MQTT Configuration
9 WIFI_SSID = "Don't"
10 WIFI_PASSWORD = "tryit9000"
```

The Shell window at the bottom displays the output of the script, showing a series of JSON messages sent by the ESP32 Node 2. Each message contains sensor data: temperature, distance, moisture, light, and battery level. The messages are formatted as follows:

```
ESP2 Sending: {"temp": 25.27, "dist": 76.63, "moist": 89.53, "light": 38.76, "batt": 70.26}
ESP2 Sending: {"temp": 25.56, "dist": 135.2, "moist": 62.08, "light": 37.59, "batt": 15.83}
ESP2 Sending: {"temp": 22.21, "dist": 170.39, "moist": 45.84, "light": 63.56, "batt": 0.4}
ESP2 Sending: {"temp": 36.29, "dist": 54.67, "moist": 74.42, "light": 18.46, "batt": 74.32}
ESP2 Sending: {"temp": 28.58, "dist": 99.62, "moist": 67.22, "light": 11.32, "batt": 0.65}
ESP2 Sending: {"temp": 38.4, "dist": 109.74, "moist": 87.59, "light": 43.76, "batt": 61.91}
ESP2 Sending: {"temp": 35.5, "dist": 91.88, "moist": 34.83, "light": 55.98, "batt": 58.03}
ESP2 Sending: {"temp": 22.75, "dist": 51.45, "moist": 57.33, "light": 76.41, "batt": 23.14}
ESP2 Sending: {"temp": 25.14, "dist": 151.47, "moist": 57.03, "light": 95.37, "batt": 90.84}
ESP2 Sending: {"temp": 22.91, "dist": 165.47, "moist": 45.99, "light": 65.35, "batt": 44.32}
ESP2 Sending: {"temp": 28.61, "dist": 76.09, "moist": 48.21, "light": 29.73, "batt": 35.68}
ESP2 Sending: {"temp": 34.71, "dist": 50.56, "moist": 77.29, "light": 5.7, "batt": 37.43}
ESP2 Sending: {"temp": 27.06, "dist": 146.22, "moist": 74.78, "light": 45.16, "batt": 7.76}
ESP2 Sending: {"temp": 33.82, "dist": 60.06, "moist": 77.02, "light": 67.15, "batt": 30.87}
ESP2 Sending: {"temp": 32.61, "dist": 162.1, "moist": 63.67, "light": 58.64, "batt": 11.97}
ESP2 Sending: {"temp": 22.81, "dist": 123.82, "moist": 39.67, "light": 9.87, "batt": 34.9}
ESP2 Sending: {"temp": 23.48, "dist": 159.18, "moist": 81.48, "light": 89.7, "batt": 39.03}
ESP2 Sending: {"temp": 39.46, "dist": 83.78, "moist": 59.48, "light": 0.5, "batt": 88.54}
ESP2 Sending: {"temp": 26.46, "dist": 199.33, "moist": 74.38, "light": 82.71, "batt": 15.94}
ESP2 Sending: {"temp": 36.75, "dist": 149.95, "moist": 54.52, "light": 23.42, "batt": 71.56}
ESP2 Sending: {"temp": 36.14, "dist": 47.18, "moist": 36.26, "light": 28.83, "batt": 64.01}
```

VS CODE



The screenshot shows the VS Code IDE interface. The main editor window displays a Python script named `espland2to_server.py` with the following code:

```
45 print(f"[!] Error parsing incoming data: {e}")
46
47 # Setup MQTT client
48 client = mqtt.Client()
49 client.on_connect = on_connect
50 client.on_message = on_message
51
52 print("[*] connecting to broker...")
53 client.connect(MQTT_BROKER, MQTT_PORT, 60)
54
55 try:
56     client.loop_forever()
57 except KeyboardInterrupt:
58     print("[*] Disconnecting. Project stopped.")
59     client.disconnect()
```

The Terminal window at the bottom displays the output of the script, showing a series of JSON messages sent by the ESP32 Node 2. Each message contains sensor data: temperature, distance, moisture, light, and battery level. The messages are formatted as follows:

```
===== DATA FROM ESP32 NODE 1 =====
Temperature (t1): 21.24 °C | Distance (d1): 38.8 cm
Moisture (m1) : 56.35 % | Light (l1) : 37.74 %
Battery (b1) : 39.6 %

===== DATA FROM ESP32 NODE 2 =====
Temperature (t2): 34.58 °C | Distance (d2): 38.16 cm
Moisture (m2) : 42.91 % | Light (l2) : 49.46 %
Battery (b2) : 13.95 %
```